МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
Кафедра «Интеллектуальные информационные технологии»

ЛАБОРАТОРНАЯ РАБОТА №3

по дисциплине «Основы машинного обучения»

Тема: «Сравнение алгоритмов классификации: k-NN, деревья решений, SVM»

Выполнил:
студент 3 курса
группы АС-67
Никитин В. Н.

Проверил:
Крощенко А. А.

Брест 2026

Цель: на практике сравнить работу нескольких алгоритмов классификации, таких как метод k-ближайших соседей (k-NN), деревья решений и метод опорных векторов (SVM). Научиться подбирать гиперпараметры моделей и оценивать их влияние на результат.

Общее задание:

1. Загрузить датасет по варианту;
2. Разделить данные на обучающую и тестовую выборки;
3. Обучить на обучающей выборке три модели: k-NN, Decision Tree и SVM;
4. Для модели k-NN исследовать, как меняется качество при разном количестве соседей (k);
5. Оценить точность каждой модели на тестовой выборке;
6. Сравнить результаты, сделать выводы о применимости каждого метода для данного набора данных.

Используемые инструменты: Python, Pandas, Matplotlib, scikit-learn, Orange Data Mining.

Задания по вариантам:

Вариант 8

• Seeds

• Классифицировать семена на три сорта пшеницы (Kama, Rosa, Canadian) на основе их геометрических параметров

• Задания:

1. Загрузите и стандартизируйте данные;
2. Разделите выборку на обучающую и тестовую;
3. Обучите три классификатора;
4. Сравните общую точность (accuracy) всех трех моделей;
5. Визуализируйте данные в 2D (например, с помощью PCA), раскрасив точки в соответствии с предсказаниями лучшей модели.

Код:

```
import os

import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.decomposition import PCA
```

```python
from sklearn.metrics import accuracy_score, confusion_matrix,
ConfusionMatrixDisplay

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
CLASS_NAMES = ['Kama', 'Rosa', 'Canadian']
COLORS = ['tab:blue', 'tab:orange', 'tab:green']

# 1. Загрузка данных
df = pd.read_csv(os.path.join(BASE_DIR, 'Seed_Data.csv'))

print("Первые 5 строк:")
print(df.head())
print(f"\nРазмер датасета: {df.shape}")
print(f"Пропусков: {df.isnull().sum().sum()}")
print("\nРаспределение классов:")
print(df['target'].map(dict(enumerate(CLASS_NAMES))).value_counts())
print("\nОписательная статистика:")
print(df.drop('target', axis=1).describe().round(3))

X = df.drop('target', axis=1)
y = df['target']

# 2. Разделение на обучающую и тестовую выборки
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
print(f"\nОбучающая выборка: {len(X_train)}, тестовая: {len(X_test)}")

# Стандартизация (параметры считаются только по обучающей выборке)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
print("\nСредние после стандартизации (обучающая выборка):",
      X_train_scaled.mean(axis=0).round(3))
print("СКО после стандартизации (обучающая выборка):",
      X_train_scaled.std(axis=0).round(3))

# Подбор k для k-NN (кросс-валидация на обучающей выборке)
k_values = range(1, 31)
cv_scores = []
for k in k_values:
    cv_scores.append(cross_val_score(KNeighborsClassifier(n_neighbors=k),
                                     X_train_scaled, y_train, cv=5).mean())
best_k = list(k_values)[cv_scores.index(max(cv_scores))]
print(f"\nОптимальное k для k-NN (по кросс-валидации): {best_k}, "
      f"CV accuracy = {max(cv_scores):.4f}")

plt.figure(figsize=(10, 5))
plt.plot(k_values, cv_scores, marker='o')
plt.axvline(best_k, color='red', linestyle='--', label=f'Оптимальное k =
{best_k}')
plt.title('Точность k-NN (кросс-валидация) в зависимости от k')
plt.xlabel('k (количество соседей)')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout()
plt.savefig(os.path.join(BASE_DIR, 'knn_k.png'), dpi=150)
plt.show()

# 3. Обучение трёх классификаторов
knn = KNeighborsClassifier(n_neighbors=best_k)
dt = DecisionTreeClassifier(random_state=42)
svm = SVC(random_state=42)
```

```
knn.fit(X_train_scaled, y_train)
dt.fit(X_train_scaled, y_train)
svm.fit(X_train_scaled, y_train)

y_pred_knn = knn.predict(X_test_scaled)
y_pred_dt = dt.predict(X_test_scaled)
y_pred_svm = svm.predict(X_test_scaled)

# 4. Сравнение точности
acc_knn = accuracy_score(y_test, y_pred_knn)
acc_dt = accuracy_score(y_test, y_pred_dt)
acc_svm = accuracy_score(y_test, y_pred_svm)

print(f"\nAccuracy k-NN (k={best_k}): {acc_knn:.4f}")
print(f"Accuracy Decision Tree: {acc_dt:.4f}")
print(f"Accuracy SVM: {acc_svm:.4f}")

print("\nСредняя точность по 5-кратной кросс-валидации (вся выборка):")
X_all_scaled = StandardScaler().fit_transform(X)
for name, model in [(f'k-NN (k={best_k})',
KNeighborsClassifier(n_neighbors=best_k)),
                    ('Decision Tree',
DecisionTreeClassifier(random_state=42)),
                    ('SVM', SVC(random_state=42))]:
    scores = cross_val_score(model, X_all_scaled, y, cv=5)
    print(f"{name}: {scores.mean():.4f} ± {scores.std():.4f}")

results = {f'k-NN (k={best_k})': acc_knn, 'Decision Tree': acc_dt, 'SVM':
acc_svm}
predictions = {f'k-NN (k={best_k})': y_pred_knn, 'Decision Tree': y_pred_dt,
'SVM': y_pred_svm}

plt.figure(figsize=(8, 5))
bars = plt.bar(results.keys(), results.values(), color=['steelblue',
'seagreen', 'darkorange'])
for bar, acc in zip(bars, results.values()):
    plt.text(bar.get_x() + bar.get_width() / 2, acc + 0.005, f'{acc:.3f}',
ha='center')
plt.ylim(0.7, 1.02)
plt.title('Сравнение точности моделей на тестовой выборке')
plt.ylabel('Accuracy')
plt.grid(axis='y', linestyle='--', alpha=0.6)
plt.tight_layout()
plt.savefig(os.path.join(BASE_DIR, 'models_comparison.png'), dpi=150)
plt.show()

fig, axes = plt.subplots(1, 3, figsize=(15, 4.5))
for ax, (name, y_pred) in zip(axes, predictions.items()):
    cm = confusion_matrix(y_test, y_pred, labels=[0, 1, 2])
    ConfusionMatrixDisplay(cm, display_labels=CLASS_NAMES).plot(ax=ax,
cmap=plt.cm.Blues, colorbar=False)
    ax.set_title(f'{name}: {results[name]:.3f}')
plt.tight_layout()
plt.savefig(os.path.join(BASE_DIR, 'confusion_matrices.png'), dpi=150)
plt.show()

best_model = max(results.items(), key=lambda x: x[1])[0]
print(f"\nСамый эффективный метод на данном датасете - {best_model}")

# 5. Визуализация в 2D с помощью PCA
pca = PCA(n_components=2)
X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)
print(f"\nPCA: доля объяснённой дисперсии PC1 =
```

```
{pca.explained_variance_ratio_[0]:.3f}, "
      f"PC2 = {pca.explained_variance_ratio_[1]:.3f}, "
      f"сумма = {pca.explained_variance_ratio_.sum():.3f}")

y_pred_best = predictions[best_model]
errors = y_pred_best != y_test.values

fig, axes = plt.subplots(1, 2, figsize=(15, 6), sharex=True, sharey=True)
for cls in range(3):
    axes[0].scatter(X_test_pca[y_test.values == cls, 0],
X_test_pca[y_test.values == cls, 1],
                    color=COLORS[cls], label=CLASS_NAMES[cls], s=60,
alpha=0.8)
    axes[1].scatter(X_test_pca[y_pred_best == cls, 0], X_test_pca[y_pred_best
== cls, 1],
                    color=COLORS[cls], label=CLASS_NAMES[cls], s=60,
alpha=0.8)
axes[1].scatter(X_test_pca[errors, 0], X_test_pca[errors, 1], s=200,
facecolors='none',
                edgecolors='red', linewidths=2, label='Ошибка модели')
axes[0].set_title('Тестовая выборка: истинные сорта')
axes[1].set_title(f'Тестовая выборка: предсказания лучшей модели
({best_model})')
for ax in axes:
    ax.set_xlabel(f'PC1 ({pca.explained_variance_ratio_[0]:.1%} дисперсии)')
    ax.legend()
    ax.grid(True, linestyle='--', alpha=0.6)
axes[0].set_ylabel(f'PC2 ({pca.explained_variance_ratio_[1]:.1%} дисперсии)')
plt.tight_layout()
plt.savefig(os.path.join(BASE_DIR, 'pca_best_model.png'), dpi=150)
plt.show()

# Вся выборка в пространстве PCA, раскрашенная предсказаниями лучшей модели
best = {'k-NN': knn, 'Decision Tree': dt, 'SVM': svm}[best_model.split('
(')[0]]
X_all_pca = pca.transform(scaler.transform(X))
y_all_pred = best.predict(scaler.transform(X))
plt.figure(figsize=(9, 7))
for cls in range(3):
    mask = y_all_pred == cls
    plt.scatter(X_all_pca[mask, 0], X_all_pca[mask, 1], color=COLORS[cls],
                label=f'{CLASS_NAMES[cls]} (предсказано)', alpha=0.7)
wrong = y_all_pred != y.values
plt.scatter(X_all_pca[wrong, 0], X_all_pca[wrong, 1], s=160,
facecolors='none',
            edgecolors='red', linewidths=2, label='Ошибка модели')
plt.title(f'Все семена в 2D (PCA), цвет — предсказание {best_model}')
plt.xlabel('PC1')
plt.ylabel('PC2')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout()
plt.savefig(os.path.join(BASE_DIR, 'pca_all.png'), dpi=150)
plt.show()
```

Результаты выполнения программы:

```
Первые 5 строк:
       A      P       C     LK     WK  A_Coef    LKG  target
0  15.26  14.84  0.8710  5.763  3.312   2.221  5.220       0
1  14.88  14.57  0.8811  5.554  3.333   1.018  4.956       0
2  14.29  14.09  0.9050  5.291  3.337   2.699  4.825       0
3  13.84  13.94  0.8955  5.324  3.379   2.259  4.805       0
```

```
4  16.14  14.99  0.9034  5.658  3.562   1.355  5.175       0
```

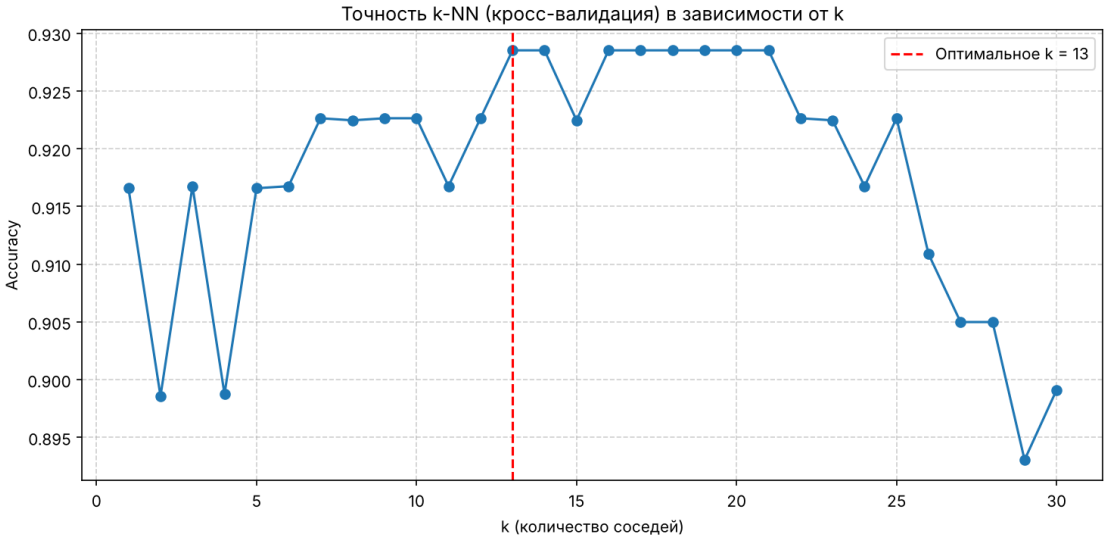
```
Размер датасета: (210, 8)
Пропусков: 0
```

```
Распределение классов:
target
Kama        70
Rosa        70
Canadian    70
Name: count, dtype: int64
```

```
Описательная статистика:
                A           P           C          LK          WK    A_Coef         LKG
count  210.000000  210.000000  210.000000  210.000000  210.000000  210.000000  210.000000
mean    14.847524   14.559286    0.870999    5.628533    3.258605    3.700201    5.408071
std      2.909699    1.305959    0.023629    0.443063    0.377714    1.503557    0.491480
min     10.590000   12.410000    0.808000    4.899000    2.630000    0.765100    4.519000
25%     12.270000   13.450000    0.856975    5.262250    2.944000    2.561500    5.045000
50%     14.355000   14.320000    0.873450    5.523500    3.237000    3.599000    5.223000
75%     17.305000   15.715000    0.887775    5.979750    3.561750    4.768750    5.877000
max     21.180000   17.250000    0.918300    6.675000    4.033000    8.456000    6.550000
```

```
Обучающая выборка: 168, тестовая: 42
```

```
Средние после стандартизации (обучающая выборка): [-0. -0.  0.  0. -0.  0.  0.]
СКО после стандартизации (обучающая выборка): [1. 1. 1. 1. 1. 1. 1.]
```

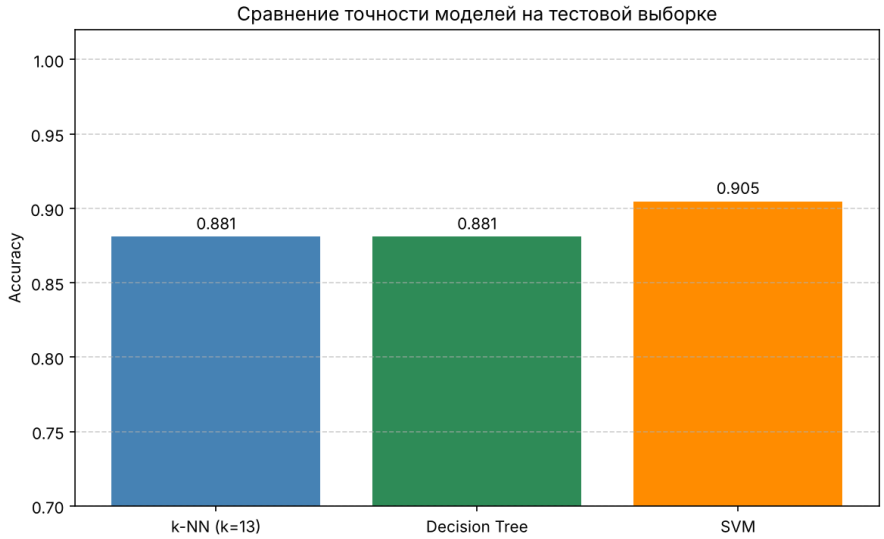


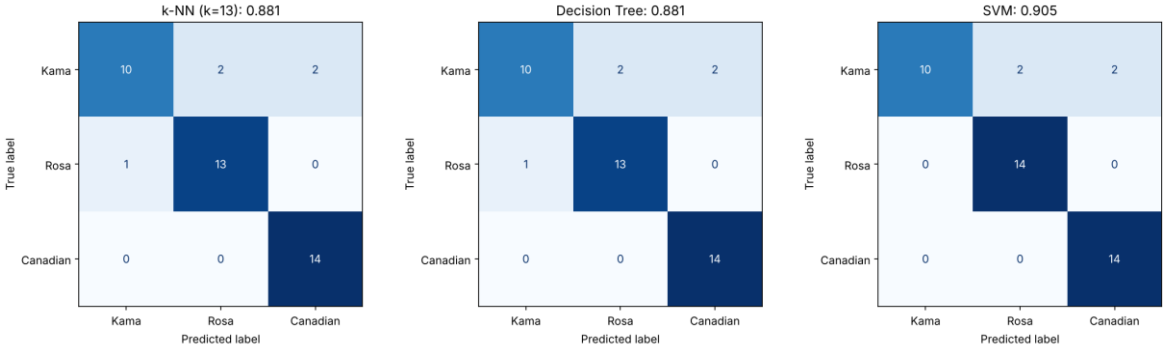

```
Оптимальное k для k-NN (по кросс-валидации): 13, CV accuracy = 0.9285
```

```
Accuracy k-NN (k=13): 0.8810
Accuracy Decision Tree: 0.8810
Accuracy SVM: 0.9048
```

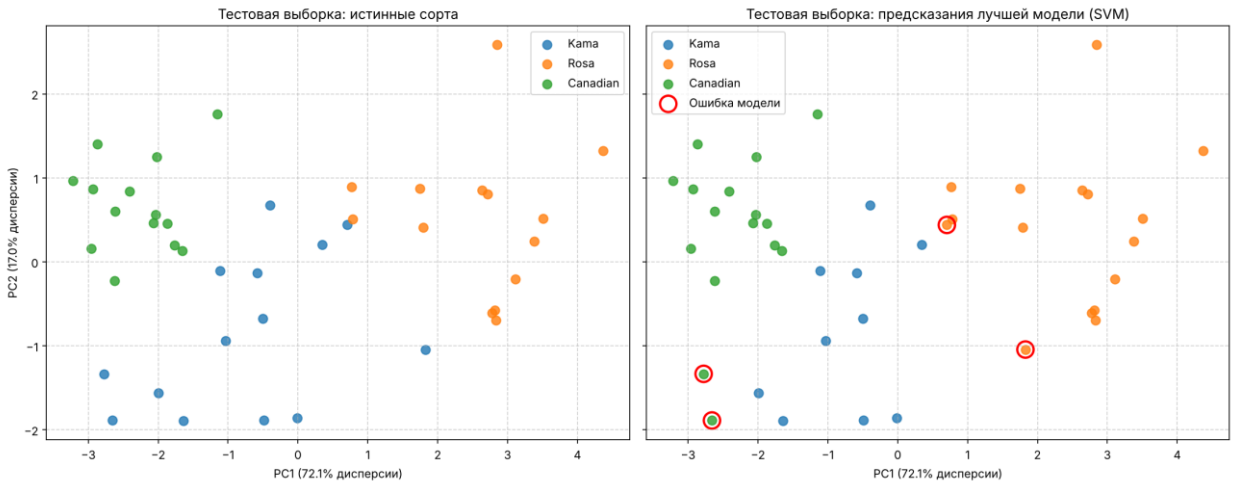
```
Средняя точность по 5-кратной кросс-валидации (вся выборка):
k-NN (k=13): 0.9190 ± 0.0683
Decision Tree: 0.8857 ± 0.0410
SVM: 0.9143 ± 0.0777
```

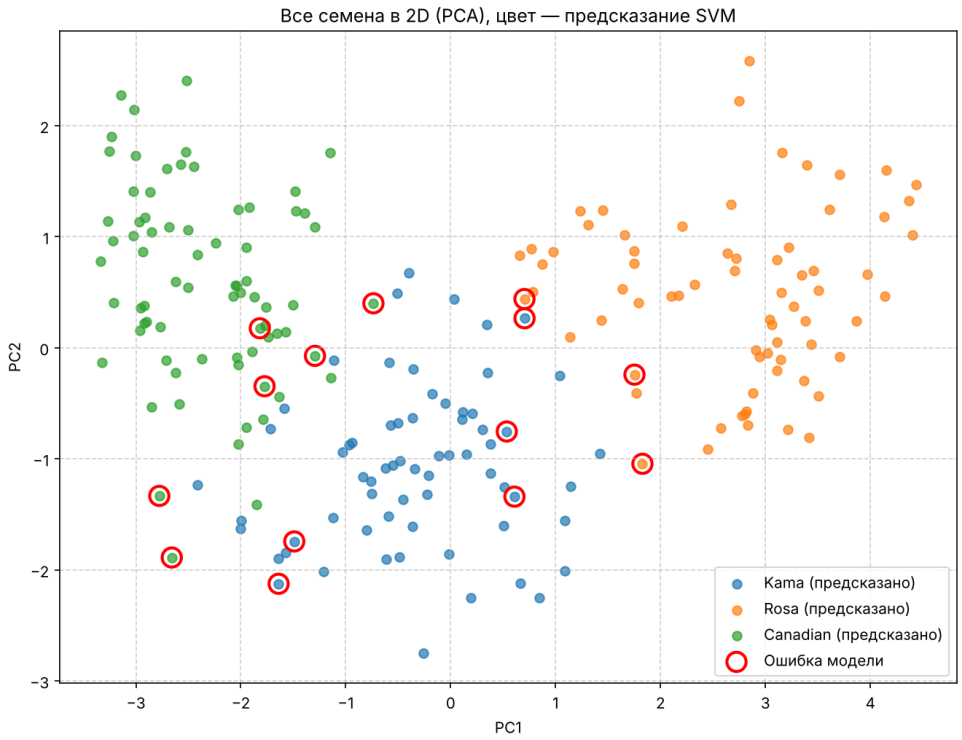
```
Самый эффективный метод на данном датасете - SVM
```





PCA: доля объяснённой дисперсии PC1 = 0.721, PC2 = 0.170, сумма = 0.891




Все семена в 2D (PCA), цвет — предсказание SVM
PC1
PC2
Kama (предсказано)
Rosa (предсказано)
Canadian (предсказано)
Ошибка модели